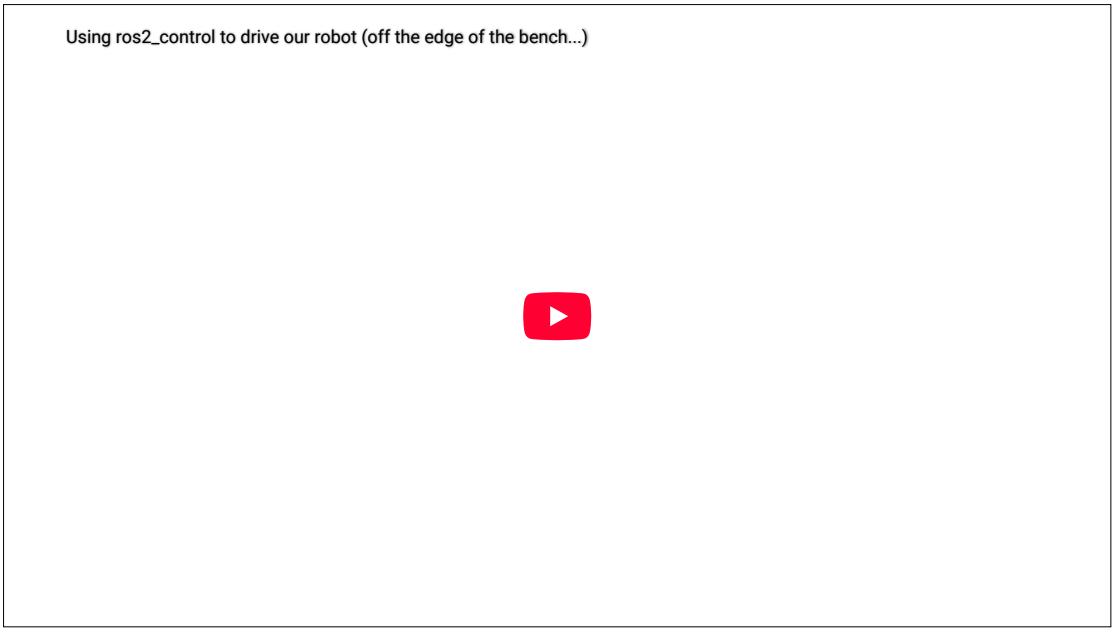
ros2_control on the real robot



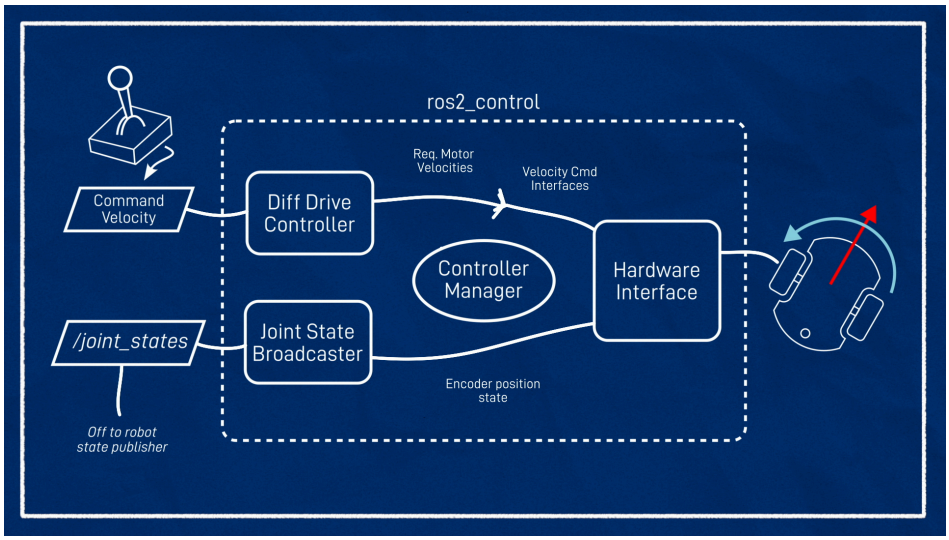
In the last tutorial we used ros2_control to drive our robot in Gazebo, but this time we're going to step things up and drive our robot in the real world. It won't make much sense if you haven't read the last tutorial, so make sure you do it first!

Before we dive into the code, let's quickly recap how ros2_control will work with our robot. At one end, we have a body velocity request, that's the velocity we want the robot to be moving through space, and we call this the *command velocity*. This could come from an operator, or something like a navigation stack. At the other end, we've got the physical robot, with two drive wheels, each of which are velocity controlled.

Although the command velocity is going to be a ROS topic of type `Twist` (or `TwistStamped` as we'll see in the next video), that is a 6-dimensional velocity but since this particular robot's motion is constrained to move forwards and backwards in X, and rotate around Z, we'll only use two of the values (`linear.x` and `angular.z`).

The tool we're going to use to link up this command velocity to the actual motor velocities is ros2_control. This little ecosystem consists of three main parts:

- The `diff_drive_controller` plugin, provided by `ros2_controllers`, which turns our command velocity into abstract wheel velocities
- The hardware interface plugin, provided by us, which turns abstract wheel velocities into signals for the motor controller (e.g. via serial commands)
- The controller manager, provided by ros2_control, in this case using the `ros2_control_node`, which links everything together



In addition to this, we have a controller called a joint state broadcaster. All this one does is read the motor encoder positions (provided by the hardware interface) and publish them to the `/joint_states` topic for robot state publisher to generate the wheel transforms. For this kind of robot it's not strictly necessary but it makes the system "complete".

In the previous tutorial we got all this working with a simulated robot, and a Gazebo-integrated controller manager, but this time we'll be doing it with a regular controller manager, our own hardware interface, on a real robot!

Getting the hardware interface

For this to work, we need a hardware interface for our robot. I've written one that you can use if you are using the same Arduino motor control code that I am, otherwise you'll need to find one online or write your own. Here's an example of one I found (have not tested) for Dynamixel motors.

> Writing your own hardware interface is out of the scope of this tutorial, but if people are interested let me know, because they changed the API after foxy, and so I need to rewrite mine for people on newer distros, and that might be a good opportunity to talk through it.

My hardware interface is called `diffdrive_arduino`, and all it does is expose the two velocity command interfaces (one for each motor), and the velocity and position state interfaces. Underneath, it's simply just sending those "m" and "e" commands over serial that we saw in the motor tutorial (*which never got a written version...oops!*). Remember, the `diff_drive_controller` will be handling the actual control allocation.

> It would be nice to be able to just `sudo apt install diffdrive arduino` but unfortunately we can't do that just yet. Right now my driver depends on a great serial library which was never officially updated and released for ROS 2. There is a source port though, so we're going to have to add it to our workspace and compile it ourselves. At some point I might rewrite my code to use a different library which IS available, but I did like that one so William or Scott if you're listening, please consider a ros 2 release. If that ever happens or I rewrite it, I'll release mine as well so that you can install it all through apt.

To get the hardware interface we'll clone from my GitHub into our workspace and build it from source. It also needs a copy of the serial library it depends on, and to keep things simple I've made my own fork (of a different fork). So let's hop onto the Pi and clone those into our workspace, then build them.

```
cd robot_ws/src
git clone https://github.com/joshnewans/serial
git clone https://github.com/joshnewans/diffdrive_arduino
cd ..
colcon build --symlink-install
```

Remember if you haven't cloned your main project package (e.g. `articubot_one` for me) onto the robot yet to do that one as well, and if you *have* already cloned it you'll want to pull the latest changes (from the last tutorial) from git.

Update the URDF

Now we need to update our URDF file to reflect the new hardware interface. Remember that we are editing these files **on the Raspberry Pi**.

> **TIP**
> To edit files on the Pi I like to use VS Code Remote Development. You can also just do it in the terminal with your favourite command line editor, or on your development machine and sync the files across to test them.

Let's open up the `ros2_control.xacro` file we made in the last video. Here we will find the `<ros2_control>` block we wrote in the last tutorial to describe our hardware interface when we were using Gazebo, but now we need one for the real system.

The steps to do that are:

- Make a copy of the Gazebo one and comment out the original. We'll come back to it later so we can toggle between them.
- Change the name (e.g. to `"RealRobot"`).
- Change the plugin to `diffdrive_arduino/DiffDriveArduino`
- Add the parameters for the hardware interface.

Hardware interface plugins can expose a bunch of parameters for us to set. In this case we'll have the following:

- Left and right wheel joint names

- Arduino loop rate
- Serial port
- Baud rate
- Comms timeout
- Encoder counts per revolution

You'll need to set the parameters appropriately for your system. The result should look something like:

```
<ros2_control name="RealRobot" type="system">
    <hardware>
        <plugin>diffdrive_arduino/DiffDriveArduino</plugin>
        <param name="left_wheel_name">left_wheel_joint</param>
        <param name="right_wheel_name">right_wheel_joint</param>
        <param name="loop_rate">30</param>
        <param name="device">/dev/ttyUSB0</param>
        <param name="baud_rate">57600</param>
        <param name="timeout">1000</param>
        <param name="enc_counts_per_rev">3436</param>
    </hardware>
    <!-- Note everything below here is the same as the Gazebo one -->
    <joint name="left_wheel_joint">
        <command_interface name="velocity">
            <param name="min">-10</param>
            <param name="max">10</param>
        </command_interface>
        <state_interface name="velocity"/>
        <state_interface name="position"/>
    </joint>
    <joint name="right_wheel_joint">
        <command_interface name="velocity">
            <param name="min">-10</param>
            <param name="max">10</param>
        </command_interface>
        <state_interface name="velocity"/>
        <state_interface name="position"/>
    </joint>
</ros2_control>
```

Launch file

Now we need a launch file. We'll start by making a copy of `launch_sim.launch.py` and call it `launch_robot.launch.py`. There are a few things we'll want to change in here.

Add the controller manager node

We'll get rid of the Gazebo launch and spawner since we don't need them any more, but something we will need is a controller manager. Remember that last time the Gazebo plugin was running a controller manager for us, but now we need to start one ourselves. This starts simply enough, we can copy one of our spawners, change the name of the variable, and also change the executable to `ros2_control_node`. We'll also leave a spot for the parameters we're about to fill in.

```
# Delete the Gazebo stuff and replace it with this
controller_manager = Node(
        package='controller_manager',
        executable='ros2_control_node',
        parameters=[XXXXXXXXX],
        )
```

You might recall from the last video that the two things the controller manager needs are the robot description (the URDF), and controller parameters.

Getting the robot description

The robot description we're going to get in a bit of an odd way - it was originally sent as a parameter to `robot_state_publisher` in `rsp.launch.py`, so we need to get it back out. Unfortunately the controller manager can't just read it from the `/robot_description` topic the way the `gazebo_ros2_control` plugin does (though I'm not sure why!).

To do this we'll use a `Command` substitution. This will be evaluated at the time when it goes to run the controller manager, and it will reqest the contents fo the `robot_description` parameter from the `robot_state_publisher` node.

```
robot_description = Command(['ros2 param get --hide-type /robot_state_publisher robot_description'])
```

Getting the controller params

For the controller parameters we just need the path to the params file, and for that we can can use the same `os.path.join` function we have in other places.

```
controller_params = os.path.join(
        get_package_share_directory('articubot_one'), # <-- Replace with your package name
        'config',
        'my_controllers.yaml'
        )
```

Now we can update that `parameters` line from before, to include our robot description and controller parameters:

```
parameters=[{'robot_description': robot_description},
                controller_params],
```

Don't use sim time

Because we're in the real world now we want to make sure that we set `use_sim_time` to false in both:

- Our launch file `launch_robot.launch.py`, where we include the robot state publisher launch (`rsp.launch.py`)
- Our parameters file `my_controller_params.yaml` (in this case, just remove the line rather than explicitly setting it to false)

Again, this will cause problems for the Gazebo simulation but we'll fix that up later.

Delaying our nodes

One problem we'll run into is that we need the robot state publisher to have finished starting up before we run the `ros2 param get` command, or else we'll fail to start the controller manager and nothing will work. The simplest (though probably not the best) way to do this is to simply add a time delay. The example below uses 3 seconds which should be enough.

```
# At the top with our imports
from launch.actions import IncludeLaunchDescription, TimerAction
...
# Directly below the controller manager node
delayed_controller_manager = TimerAction(period=3.0,actions=[controller_manager])
```

We can also then delay the controller spawners until the controller manager has started. They do have a decent timeout anyway and so will probably be fine, but chaining it together like this is a bit neater.

```
# At the top with our imports
from launch.actions import RegisterEventHandler
from launch.event_handlers import OnProcessStart
...
# Directly below the current spawners
delayed_diff_drive_spawner = RegisterEventHandler(
        event_handler=OnProcessStart(
            target_action=controller_manager,
            on_start=[diff_drive_spawner],
        )
    )
delayed_joint_broad_spawner = RegisterEventHandler(
        event_handler=OnProcessStart(
            target_action=controller_manager,
            on_start=[joint_broad_spawner],
        )
    )
```

Finally, we want to replace the Gazebo entries with `delayed_controller_manager` in our `LaunchDescription` at the end.

```
return LaunchDescription([
    rsp,
    delayed_controller_manager,
    delayed_diff_drive_spawner,
    delayed_joint_broad_spawner
])
```

Ok, that *should* be all we need. Let's rerun colcon to add the new launch file, and then we can test it!

Testing it out!

Basic driving

WARNING! Make sure you prop your robot up before turning on the controllers for the first time, in case it sends incorrect values to the motors and sends the robot flying!

As a first test, we want to:

- Run our `launch_robot.launch.py` script on the Pi, via SSH
- Start RViz on our development machine, with the fixed frame set to `odom` and various displays enabled (e.g. TF, RobotModel)
- On the dev machine, run `teleop_twist_keyboard` (or your preferred teleop method) with the topics remapped like in the previous tutorial (`-r /cmd_vel:=/diff_cont/cmd_vel_unstamped`)

As we send velocity commands, we should see:

- The wheels spinning in the expected direction
- The RViz display moving as if the robot were actually driving (it doesn't know the wheels are spinning in the air!)
- All the transforms etc. are displayed with no errors

If that seems to be working then it's time to take the robot off its stand and try to drive it around! If you are finding that the wheel speeds are too fast/slow, the on-screen instructions for `teleop_twist_keyboard` show how to adjust them, however we'll look at some better approaches to teleop in the next tutorial.

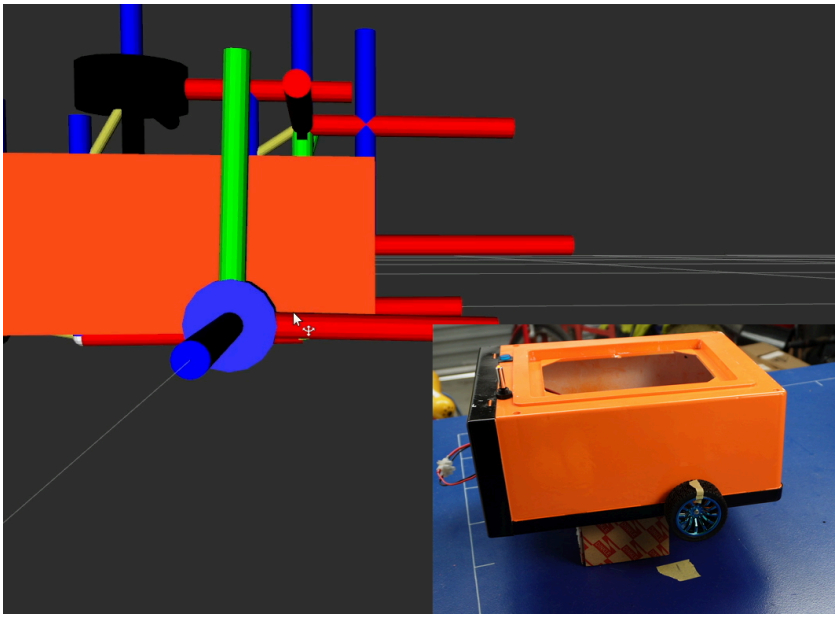
Odometry accuracy

Now we want to see how accurate our odometry is. Odometry will never be perfect, which is why we use other localisation methods such as SLAM or GPS, but it is still worth getting as good as we can. There are three easy steps we can take to assess this (ensure they are done in order as each step is affected by the previous).

1. Encoder accuracy

- Prop the robot back up.
- Restart the controller so that the wheel axes are reset in RViz and note which way they are pointing (e.g. Y is pointing up).
- Set the RViz fixed frame to something robot-fixed (e.g. `base_link`)
- Put a marker (e.g. some tape) on the wheel.
- Drive forward for a while (e.g. 30 revolutions) and stop with the marker in the same spot

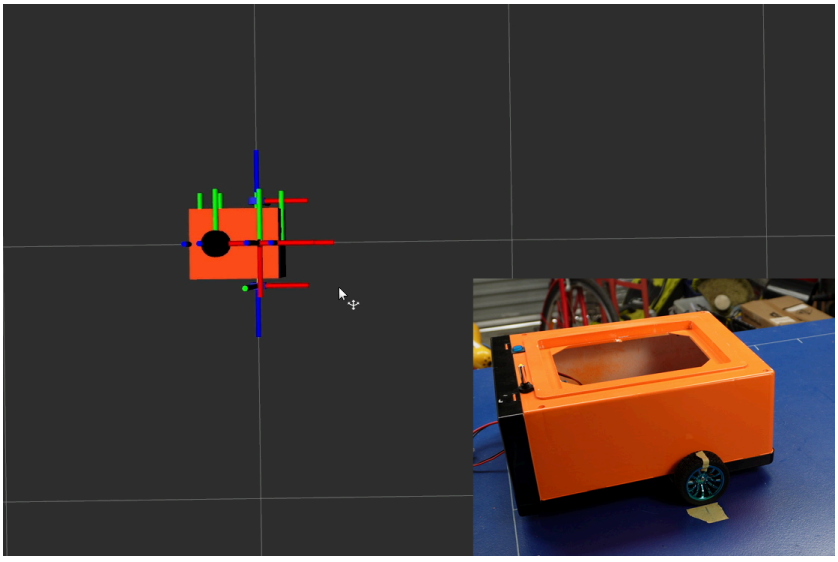
Your axes in RViz should be aligned the same as they were earlier. If it isn't, the encoder counts aren't being properly converted to angular measurements. Check that your counts-per-revolution are correct.



2. Wheel radius

- Place the robot on a large surface and note the wheel locations (e.g. with tape on the ground).
- Make another marker at a known distance ahead (e.g. 1m)
- Restart the controller so that the origin is reset in RViz (fixed frame doesn't matter too much but `odom` is best)
- Drive the robot until the wheels are aligned with the marker

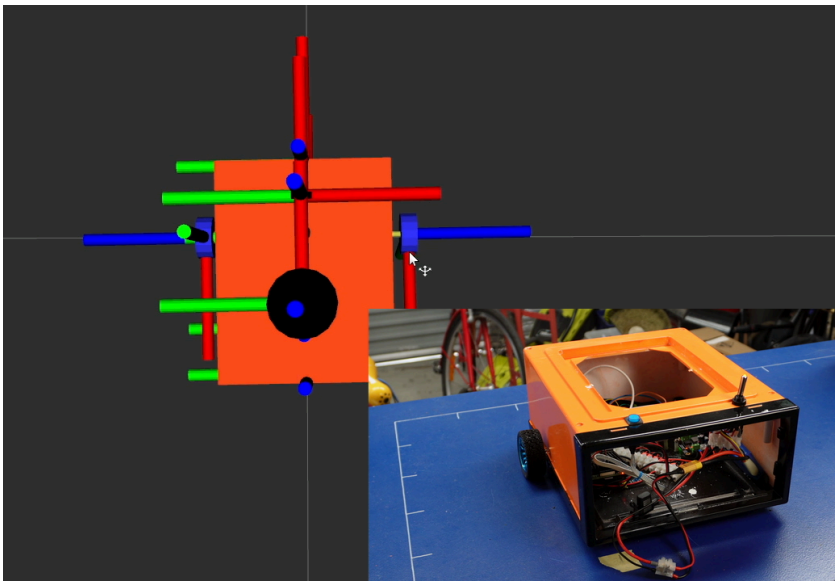
The distance travelled in RViz should be the same as in real life. If not, you need to adjust your wheel radius.



3. Wheel separation

- Place a marker on the ground at the robot wheels
- Restart the controller to reset the robot orientation in RViz
- Drive the robot one revolution so that the wheels are back where they started

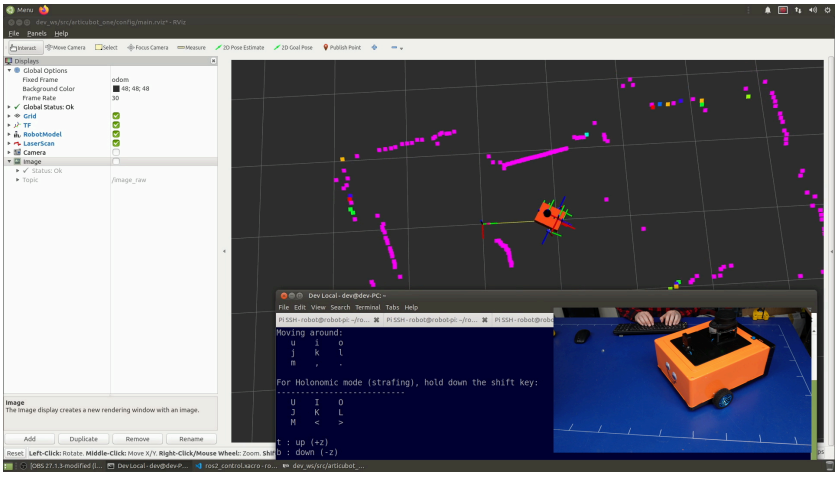
The robot should have turned exactly one revolution in RViz. If not, adjust your wheel separation.



Sensor Integration

This is also a chance to start testing our sensors (lidar, camera, and/or depth camera). We can simply run the launch files we created for these in previous tutorials.

We should see the laser data come up in RViz, and hopefully as we drive around, the detected objects should stay in approximately the same place. There will be some errors due to timing and also odometry drift.



Unbreaking Gazebo

Now that things are working, let's unbreak it for Gazebo simulations. At this point we should push our changes (on the Pi) to git, and pull them back onto the dev machine.

Xacro switch

The first thing we'll want to do is to put that block we commented out in the xacro onto a toggle. These steps are very similar to what we did in the extra part of the previous tutorial:

- Add another argument line in our main `robot.urdf.xacro` and call it `sim_mode`, defaulted to `false`.
- In `ros2_control.xacro`, wrap the "real version" in `<xacro:unless value="$(arg sim_mode)">`, and the Gazebo version (uncommented) in `<xacro:if value="$(arg sim_mode)">`.
- In `rsp.launch.py`, expand our xacro command to read as below. Whenever `use_sim_time` is true, we will be enabling `sim_mode`.

```
robot_description_config = Command(['xacro ', xacro_file, ' use_ros2_control:=', use_ros2_control, ' sim_mode:=', use_sim_time])
```

Now, whenever we are simulating, the Gazebo version should be enabled, and when we are running the live system the other one will be. Of course there are many `<gazebo>` tags throughout our URDF that we *could* disable when not in sim mode, but they will not be evaluated anyway so it doesn't really matter.

Controller sim time

Remember that previously we had disabled `use_sim_time` in the `my_controller_params.yaml` file? The problem is that for Gazebo we want this to be turned on. This is a bit messy, and it all comes down to the `gazebo_ros2_control` plugin, so let's swap back to `ros2_control.xacro`.

Ideally, we want to be able to enable `use_sim_time` for the plugin without having to have it inside the params file. Some ways this could possibly work are if the plugin always enabled it (since it's for Gazebo), or had a way to specify single parameters. Right now though, the only option is to specify *multiple parameter files* and it will combine the parameters from all of them.

So we can create a new params file (e.g. `config/gaz_ros2_ctl_sim.yaml`) containing only:

```
controller_manager:
  ros__parameters:
    use_sim_time: true
```

Then, in `ros2_control.xacro` we can add it to the plugin include, so that we have:

```
<gazebo>
    <plugin filename="libgazebo_ros2_control.so" name="gazebo_ros2_control">
        <parameters>$(find articubot_one)/config/my_controllers.yaml</parameters>
        <parameters>$(find articubot_one)/config/gaz_ros2_ctl_sim.yaml</parameters>
    </plugin>
</gazebo>
```

So on the real robot it'll just read the main params file, and on Gazebo it'll do both, and so have sim time enabled.

> Note: As of writing this tutorial, this functionality has only just been added and has not been pushed to the package repositories. Until then, you'll need to build `gazebo_ros2_control` yourself by running `git clone -b foxy https://github.com/ros-controls/gazebo_ros2_control` in your dev workspace.

Conclusion

Now we can drive our robot around which is a huge step forward, and hopefully it's starting to feel like a real robot. In the next tutorial we'll look deeper at *teleoperation* (remote control), including how to control our robot using a game controller, or a smartphone.